

Daily Scholar Papers Report — 2026-08-18

2026-08-18

Daily Scholar Papers Report — 2026-08-18

[Download PDF](#)

Window covered: 2026-08-17 → 2026-08-18 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

One alert thread, six candidates, three deep-reads — and for the first time in four days the fetch layer cooperated, so all three are full-text reads with the papers' own numbers rather than abstract-level pointers. Three papers arrived independently, from three different subfields, and made structurally the same argument: **the expensive stage of your pipeline is the one nobody bothered to measure, and once you measure it the fix is usually cheap.**

SyzMini states the thesis most plainly because it is the only one of the three whose contribution *is* a measurement. Kernel fuzzers minimise every interesting program before adding it to the corpus, and everyone knew this cost something. Nobody had published what. The answer is that **57.5 %** of all program executions in a 48-hour Syzkaller campaign are spent in minimisation, peaking at **68.1 % at the 4.5-hour mark** — a fuzzer that spends more than half its budget shrinking inputs rather than exploring. The obvious response, deleting the stage, is worse: coverage falls **27.5 %** and bug count **40.4 %**. So minimisation is simultaneously load-bearing and ruinous, and the paper's two optimisations cut its cost by **60.7 %** while *improving* coverage **12.5 %** and finding **1.7–2.0x** more unique bugs, including **13 previously unknown** ones. The share of executions spent minimising drops from **48.2 % to 11.5 %** on Linux 5.15. That is not a tuning result; it is close to doubling the useful throughput of a widely deployed fuzzer by looking at where the budget went.

ConfFuzz reaches the same place from the direction of the fuzz driver. Cloud systems expose hundreds to thousands of configuration parameters — **2,771** in the HDFS module alone — and the average unit test touches **15.14 %** of them, with 90 % of tests in three Hadoop projects touching under 10 %. The prior state of the art drove *system* tests, at a measured **0.0371 executions per second**, roughly 27 seconds each. ConfFuzz drives existing *unit* tests instead, at **20.57 executions per second** — about **550x** faster — which is the number that makes greybox feedback viable in this domain at all. On top of that it adds the actual technique: mutate only the parameters the parent seed was *observed to read*, re-measured every execution because flipping one parameter opens paths that read new ones. The component ablation is clean, each design decision isolated in turn: **19.4 → 59.2 → 68.6 → 94.8** average bugs as parameter-aware mutation, then default-configuration seeding, then parameter-aware energy allocation are switched on. **125 bugs** total, **14 fixed** and **12 confirmed** by Apache developers so far, backed by five repetitions, 95 % confidence intervals, paired *t*-tests and Cohen's *d* across **48,380 VM-hours**.

CVE-Genie is the window’s most ambitious system and its most quotable ablation. It reproduces **428 of 841** real CVEs end-to-end — vulnerable environment rebuilt, working exploit synthesised, and a verifier that certifies the exploit actually triggered the vulnerability — across **267 projects, 141 CWEs and 22 languages**, at **\$2.77 per CVE**. For calibration, the manual baseline it cites spent **3,600 hours** to reproduce **368** vulnerabilities in a single project. But the number to take away is from Table 4: collapsing the eleven-stage, four-module pipeline into a single monolithic agent on the same strong model yields **0 of 15** reproductions, against **15/15** for the full system. Not degraded — zero. Removing just the critic agents gives 8/15 *and raises false reproductions by 47 %*, which is the more uncomfortable half of that result, because a false reproduction is a benchmark entry that looks fine and is wrong.

The through-line worth carrying forward is a design pattern all three share, and it is subtler than “measure first”. Each puts an admittedly imprecise component in a position where being wrong costs *time* rather than *correctness*. SyzMini’s influence relation is an unsound approximation, so a bad guess merely triggers a rollback and the sound one-by-one loop still runs. ConfFuzz’s benign-failure filter is, in the authors’ own words, “neither sound nor complete”, but it only decides what to show a human. CVE-Genie’s critics are deliberately biased toward rejection — every critic has higher TNR than TPR — so the system under-reports rather than fabricates. Imprecise analysis used for *ordering and triage*, never for *deciding*, is the reusable idea in this window.

Outstanding: 2 · Keep: 1 · Borderline High-Priority: 1

Highlighted Papers

Title	Authors	Venue	Link
CVE-Genie: An LLM-Based Multi-Agent Framework for Reproducing CVEs	S. Ullah, P. Balasubramanian, W. Guo, A. Burnett, H. Pearce, C. Kruegel, G. Vigna, G. Stringhini	CCS 2026	10.1145/3830454.3832602
ConfFuzz: Parameter-Aware Greybox Fuzzing for Configurable Cloud Systems	S. Wang, H. Wang, D. Marinov, T. Xu, Y. Zhang	ASE 2026	10.1145/3832783.3837476
Efficient Input Minimization for Kernel Fuzzing via Relation- and Type-Guided Optimization	H. Guo, H. Sun, S. Huang, T. Su, G. Pu, S. Li	ACM TOCS, 2026 (extends USENIX ATC 2025)	10.1145/3840388
Mitigating Keyword Bias in Java Vulnerability Detection through Dual-Stream CodeBERT with Security Feature Engineering	A. Khurana, T. Farasat, J. Posegga, F. Lemmerich	CEUR-WS Vol-4238, 2026	ceur-ws.org/Vol-4238/paper1.pdf

Papers

2.1 · CVE REPRODUCTION · 428 of 841 real CVEs rebuilt, exploited and verified end-to-end at \$2.77 each — and the same model as one monolithic agent scores 0/15 🍌🤔👉

CVE-Genie: An LLM-Based Multi-Agent Framework for Reproducing CVEs

Authors: Saad Ullah, Praneeth Balasubramanian, Wenbo Guo, Amanda Burnett, Hammond Pearce, Christopher Kruegel, Giovanni Vigna, Gianluca Stringhini.

Venue: CCS 2026

Affiliations. Boston University (Ullah, Stringhini), UC Santa Barbara (Balasubramanian, Guo, Kruegel, Vigna), Arizona State University (Burnett), UNSW Sydney (Pearce). Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security, The Hague, 15–19 November 2026. DOI: <https://doi.org/10.1145/3830454.3832602>. Extended version: [arXiv:2509.01835](https://arxiv.org/abs/2509.01835). Artifacts: <https://github.com/BUseclab/cve-genie>, experiment archives at <https://osf.io/dcej4/>.

Licence. CC BY 4.0, copyright held by the owner/author(s).

Provenance. Full-text read of the CCS camera-ready. Figures 5a–7a, 8 and 10 contain bar/line data whose numeric values are not recoverable from the PDF text layer; where that is the case it is stated rather than estimated.

The problem

Vulnerability research needs datasets of real bugs that come with a runnable environment and a working exploit, not just a patch diff and a label. Those are scarce because building them is manual. The paper’s calibration figures: NVD tracked more than **40,000 vulnerabilities** in **2025** alone, while a prior effort spent over **3,600 man-hours** to reproduce **368** memory vulnerabilities in Linux, and roughly **50 %** of the data in one widely used detection dataset has been shown to carry wrong labels. Recent LLM-based exploitation work operates on benchmarks of **15–40 CVEs** requiring “up to 720+ man-hours” of environment preparation.

The framing contribution is **EAGER**, a five-property specification for what CVE reproduction ought to deliver, quoted verbatim: “**Exploit Generation** – (re)creation of an exploit or PoC that reliably triggers the vulnerability in the CVE”; “**Assessment** – inclusion of a ‘verifier’ or ‘sanitizer’ capable of assessing whether the generated exploit or PoC successfully triggers the vulnerability”; “**Generalization** – reproduction of CVEs across diverse CWEs, programming languages, and software projects”; “**End-to-end Automation** – execution of all stages of CVE reproduction in a fully automated manner”; “**Rebuild project** – reconstruction of the original vulnerable environment to facilitate exploit/PoC execution.”

Defining the specification first and then using it as the axis of the related-work table is a cheap rhetorical device that does real work — it converts “prior tools each do part of the job” into a matrix in which the contribution localises itself.

Architecture

The design principle is stated as **compositional intelligence**, verbatim: “the ability to solve complex tasks by decomposing them into meaningful sub-tasks whose outputs are explicitly composed and verified to form a correct end-to-end solution.” Four modules, eleven numbered stages, every run inside

a fresh copy-on-write overlay of a single Ansible-provisioned Ubuntu 22.04 QEMU/KVM image with 2 vCPUs and 8 GB RAM, offline, with agents reaching the environment only through CLI tools.

Module	Agents	Role
Processor	Data Processor (non-LLM), Knowledge Builder	Locate vulnerable source; distil CVE description, CWEs, patch commits and advisories into a structured knowledge base serving as long-term memory
Builder	Prerequisite Developer, Setup Developer, Setup Critic	Explore read-only and plan; then execute the build; then audit the logs
Exploiter	Exploit Developer, Exploit Critic	Replicate or synthesise a PoC; then check it is not a fabricated shortcut
CTF Verifier	Verifier Developer, Flag Checker (non-LLM), Verifier Critic	Turn the CVE's success signal into a capture-the-flag challenge; run it; audit the checking logic

Two structural decisions carry most of the weight.

The first is the **read-only/write split** in the Builder. The Prerequisite Developer gets only `ls` and `get_file` and produces a plan plus an explicit definition of the “expected state” that signals a finished setup; the Setup Developer gets the write and exec tools. The stated motivations are that READMEs are frequently wrong, and that combining exploration with building overruns the context window. The ablation prices this precisely: removing the split costs only 2/15 in success but produces **27 % more tool-call limit violations**.

The second is the **CTF flag**, which is the most portable idea in the paper. Rather than writing a bespoke oracle per CVE, the system derives a CVE-specific success signal from official CVE data — a `RecursionError`, an `AddressSanitizer` crash, unauthorised access to a particular record — and wraps it in a verifier that reveals a hidden flag string only if that signal fires — the flag “is revealed only if the exploit correctly satisfies the CVE-specific success signal.” The Verifier Developer writes it in three fixed steps (pre-setup, exploit execution, post-setup), and the paper notes: “We make sure that the agent cannot modify the PoC script to avoid contamination.”

What that buys is a single uniform pass/fail interface across wildly heterogeneous vulnerability classes, a natural anti-tampering barrier, and a machine-checkable signal for downstream tooling. Any domain that needs semantically grounded correctness oracles over heterogeneous tasks can lift this directly.

Figure 4 is the compact argument for why verifiers need auditing at all: attempt 1 checks for the string `RecursionError` in the output, which is trivially spoofable and does not even confirm the vulnerable version is installed; attempt 2, after critic feedback, pins `sqlparse < 0.5.0`, runs the exploit in-process to dodge timeout artefacts, and requires the traceback to originate in `sqlparse/sql.py`.

Cost engineering

The per-module model assignment is a result, not an implementation detail. Developer agents, which burn many turns, run **o4-mini**; critics, which make one call, run **o3**. The paper reports that using reasoning-heavy models as Builder developers costs **\$10–20 per CVE** — against a whole-pipeline

average of **\$2.77**. Any agentic system with a high-turn generator and a low-turn evaluator can copy the asymmetry.

Budgets were set empirically: successful reproductions “typically required about **\$2** and **18 minutes** on average”, with the worst case at **\$6** and **45 minutes**, while failures averaged up to **\$4** and **35 minutes** — so the caps are **\$5** and **45 minutes**. Feedback iterations are capped at 1 for the Builder and Exploiter and 5 for the Verifier, because the maximum observed useful iteration count was 4, and because “fundamental issues (e.g., mock-up version of the project) persisted even after five iterations.”

Ablation (Table 4, on D_R = 15 CVEs with complete information)

Setting	Result	Secondary effect
Ideal case	15 / 15	—
No Knowledge Builder	9 / 15	↑ 30 % exploit failure
No Prerequisite Developer	13 / 15	↑ 27 % tool-call limit violations
No feedback loops	5 / 15	↓ 67 % reproduction rate
No critic agents	8 / 15	↑ 47 % false reproductions
Single monolithic agent	0 / 15	—

The bottom row is the paper’s most transferable single number. A frontier reasoning model, given the same task, the same environment and the same information but no decomposition, reproduces nothing at all. Note also the deliberate choice to run ablations on an easy subset where the ideal case is 15/15 — because the baseline saturates, every drop is unambiguously attributable to the manipulation rather than to task difficulty. That is a good ablation-design habit independent of the subject matter.

The critic row deserves separate attention. Removing critics costs 7/15 in success but increases *false* reproductions by 47 %, and false reproductions are the failure mode that actually damages a dataset, because they are indistinguishable from successes without manual audit.

Results at scale

D_L comprises **841 CVEs** published between June 2024 and May 2025, spanning **186 CWEs**, **29 languages** and **440 open-source projects**. Representativeness is argued rather than assumed: across all CVEs published in the window, roughly **55 %** fall under the MITRE top-25 CWEs and D_L matches at **55 %**; roughly **50 %** lack any PoC and D_L is at **49 %**.

428 of 841 were reproduced, across **267 projects**, **141 CWEs** and **22 languages**. Around **46 %** of successful reproductions had no prior PoC available.

The per-family breakdown is where the practical guidance lives:

Category	Success rate
CWE-1333 (ReDoS)	79 %
CWE-89 (SQL injection)	52 %
CWE-79 (XSS)	46 %
CWE-287 (auth bypass), PoC available	100 %

Category	Success rate
CWE-287 (auth bypass), no PoC	36 %
CWE-416 (use-after-free)	30 %
CWE-78 (OS command injection), no PoC	0 %
Desktop applications	18 %
Mobile	29 %
Blockchain	8 %

The CWE-287 pair is the sharpest evidence that PoC availability, not vulnerability class, is the dominant variable for whole families.

Context ablation. Systematically deleting input fields measures which artefacts actually carry signal: removing the patch commit leaves **10/15 (67 %)**, removing the advisory and PoC leaves **11/15 (73 %)**, and description-only leaves **9/15 (60 %)**. Patches matter slightly more than advisories because they localise the root cause. This methodology transfers to any system consuming a structured multi-field artefact — bug reports, issue templates, API specifications.

Audit. Both sides of the confusion matrix were sampled: **all 50** runs that passed CTF verification were confirmed correct on manual review, and among 50 sampled failures **12 false negatives** were found, correct exploits rejected for insufficient evidence in the trace. This is consistent with the deliberate design bias — every critic exhibits higher TNR than TPR.

Formal notation. The paper contains no numbered display equations and no formal mathematical characterisation. Its formal content is the EAGER prose criteria, the tables, and Python code listings. Per this digest’s formula-integrity rule, none is invented here.

Limitations

There is no dedicated limitations section; the following are distributed through §3.5 and §4.4. The sharpest is self-reported. Where no deterministic signal existed — no sanitizer crash, no exception — verification fell back on heuristic time and memory thresholds: a 3-second cutoff for one DoS CVE, a 300 MB peak-RSS ceiling for another. The authors note these are “sensitive to system load and hardware configuration”, with the consequence, in their words, that a legitimate exploit **“could pass on one machine but fail on another”**. For a dataset intended as a stable benchmark, machine-dependent verdicts across part of the corpus is a real caveat.

Of the **413** failures, **198** exhausted cost or time budgets across all three runs, with cost overruns roughly **80 %** of those, split between project builds (~41 %, mostly web applications, which depend on installation guides unreachable from the offline VM) and exploit generation (~59 %). **Seven** CVEs are structurally out of scope: four need Windows filesystem behaviour, two need macOS entitlements and XPC, one needs a full ASP.NET/IIS stack. Non-determinism means the Builder is the bottleneck, and the paper notes that memory-safety bugs can execute silently without sanitizer instrumentation, failing verification.

Why it is worth your time

Because the transferable content survives the removal of every security-specific detail. Three ideas travel: the CTF-flag construction as a universal interface for heterogeneous correctness oracles; the

asymmetric cheap-generator/expensive-critic cost architecture; and the decomposition result itself, which is the strongest published evidence that agentic pipelines are not a stylistic preference over single-agent prompting but the difference between 15/15 and 0/15.

Closing line, verbatim: “To the best of our knowledge, no other automated framework has demonstrated end-to-end CVE reproduction at this scale and cost-efficiency.”

Figure note. Figure 2, the two-page architecture-and-worked-example diagram, is the natural embed and is CC BY 4.0. It is not reproduced here — figure extraction requires local raster tooling that was unavailable this run. Attribution would be: Ullah et al., CCS 2026, CC BY 4.0.

2.2 · CONFIGURATION FUZZING · Unit tests as configuration fuzz drivers at 20.57 exec/s against system testing’s 0.0371 — about 550× — yielding 125 bugs, 14 already fixed 

ConfFuzz: Parameter-Aware Greybox Fuzzing for Configurable Cloud Systems

Authors: Shuai Wang, Hao Wang, Darko Marinov, Tianyin Xu, Yongle Zhang.

Venue: ASE 2026

Affiliations. University of Illinois Urbana-Champaign (S. Wang, Marinov, Xu), UC Berkeley (H. Wang), Purdue University (Zhang). 41st IEEE/ACM International Conference on Automated Software Engineering, Munich, 12–16 October 2026. DOI: <https://doi.org/10.1145/3832783.3837476>. Artifact: <https://doi.org/10.5281/zenodo.19248494>. Received 2026-03-26; accepted 2026-06-18.

Licence. CC BY 4.0, copyright held by the owner/author(s).

Provenance. Full-text read of the ASE camera-ready.

The problem

Cloud systems expose enormous configuration surfaces, and essentially nobody tests them. The motivating numbers: faulty configurations were reported as “the second largest cause of service disruptions” in a major Google production service, and at Meta **16 %** of service-level incidents, including significant outages, were introduced by configuration changes. Meanwhile mature systems have abundant unit tests achieving high coverage, but those tests “exercise only a small set of handpicked configurations, most often the default”.

Quantified: **on average, a unit test exercises only 15.14 % of the total parameters**, and in HCommon, MapReduce and Yarn, **90 % of unit tests exercise less than 10 %** of them. The evaluated module of HDFS alone exposes **2,771** parameters.

Prior work splits badly. The blackbox cloud-scale fuzzer uses no execution feedback and drives “a small number of manually written system tests (**2.6 per project** on average)”. The greybox option was built for command-line options, where “each application exposes only **1 to 14** options, **92.6 %** of which are booleans” — orders of magnitude smaller than a cloud configuration space. And the reason plain coverage-guided fuzzing does not close the gap is stated crisply: code coverage “does not tell which configuration parameters were exercised”, and so cannot indicate which region of the configuration space to mutate next.

The technique

Three ideas, each isolated in the ablation.

Unit tests as fuzz drivers, obtained by instrumenting the interface rather than synthesising harnesses. ConfFuzz patches the configuration-class constructor so that every configuration object a test builds is replaced by a generated one, and patches the unified GET API so every parameter read is recorded. Cost: about **half an hour per project** and **20 lines of code on average** — a minimum of **11** for the smallest subject, a maximum of **37** for the largest — at **3 %** average tracking overhead. Every existing test silently becomes a driver. A nice piece of care: the tracker records the *resolved* parameter name after alias/deprecation handling, “which is exactly the configuration parameter the test observes, rather than an unrelated alias.”

The payoff is the number that reframes the problem. ConfFuzz averages **20.57 executions per second (48.6 ms per execution)**, against a reported **0.0371 testcases per second** — about 27 seconds each — for system testing on the same class of systems. Roughly **550× faster**, which is what makes feedback-driven search viable here at all.

Parameter-aware mutation. Each seed stores a triple of configuration, code coverage, and exercised-parameter set. Mutation is restricted to the *parent seed’s* exercised set; everything else inherits the parent’s values. The essential subtlety is that this set is re-measured on every execution rather than computed once. Flipping a single parameter can steer execution down a path that reads parameters nobody had seen, so a set collected before fuzzing — the default configuration’s, say — is, in the paper’s words, “insufficient for iterative greybox fuzzing.”

Dual-signal feedback, split deliberately across two roles. At *seed admission*, a candidate is saved if it covers new branches **or** exercises a previously unseen parameter — because “a candidate may expand code coverage without exercising any previously unseen parameter”, and that novelty is invisible to branch coverage. At *scheduling*, energy is allocated by a product of normalised coverage and parameter weights. The paper is explicit about the division of labour: the saving rule “rewards marginal discoveries in the current round”, while the budgeting rule prioritises seeds in later rounds by the breadth of behaviour they already exercise.

The energy formulas, from §3.3 — note that the paper does **not** number its equations, so they are cited by section:

$$w_c(s, S) = \frac{|\text{coverage}(s)|}{\max_{s' \in S} |\text{coverage}(s')|}$$

$$w_p(s, S) = \frac{|\text{params}(s)|}{\max_{s' \in S} |\text{params}(s')|}$$

$$\text{numCandidates}(s, S) = \max(1, \lfloor b \cdot w_c(s, S) \cdot w_p(s, S) \rfloor)$$

where `coverage(s)` is the set of branches covered by seed `s`, `params(s)` the set of parameters it exercises, `S` the current seed queue, and `b` the base mutation budget (set to 50, following JQF). The multiplicative form with max-normalisation is directly portable to any fuzzer that has a second measurable breadth signal.

Algorithm 2 presents the whole thing as a diff against the standard coverage-guided loop of Algorithm 1, with the ConfFuzz-specific lines highlighted — a presentation choice worth stealing, since it makes the contribution inspectable in about thirty seconds.

Results

Nine subjects, evaluated one core module each: HCommon (892 parameters), HDFS (2,771), HBase (1,348), Alluxio (847), Hive (669), MapReduce (270), Yarn (242), Kylin (150), Zeppelin (146). **4,838 fuzz drivers, 30 minutes** each, **five repetitions**, four variants — **48,380 VM-hours**, which the paper points out is “well beyond the usual 24-hour by 10-trial fuzzer evaluations.”

Mean bugs over five runs, with each component switched on in turn:

Variant	Components	Mean bugs	Mean unique failures
ConfFuzz_Base	coverage only (reimplemented prior greybox strategy)	19.4	158.0
ConfFuzz_Track	+ parameter-aware mutation	59.2	348.0
ConfFuzz_Track+Def	+ start from default config	68.6	361.2
ConfFuzz	+ parameter-aware energy allocation	94.8	499.0

Every paired comparison is significant under a paired t -test with all 95 % confidence intervals excluding zero and large effect sizes — ConfFuzz over Base is $\Delta = +75.4$ bugs, $p = 5.7 \times 10^{-7}$, $d_z = 25.42$. Across the union of five runs the variants expose **125 unique bugs**, of which ConfFuzz finds **118**, and **38** are found by ConfFuzz alone.

Coverage: ConfFuzz achieves about **14.2 %** more branches than ConfFuzz_Track and roughly **3.2×** ConfFuzz_Base (geometric mean of per-project ratios), winning on eight of nine projects — largest gains HDFS **+52.2 %**, HBase **+40.5 %**, Hive **+17.5 %**; the single loss is Zeppelin, the smallest configuration space, at **0.4 %** below.

Parameter-space exploration: HDFS grows the exercised-parameter set to **39.5×** the initial seed against 25.6× and 25.0× for the tracking variants. Across all drivers the median exercised count grows from **22** to **35** and the mean from **118.8** to **194.2** over 30 minutes, with **60.1 %** of drivers exercising at least one new parameter.

Developer response: **77 of the 125** bugs reported so far, **26** answered, of which **14 fixed** and **12 confirmed**. Bug taxonomy: crashing with no message 41 (32.8 %), NullPointerException 25 (20.0 %), semantic 23 (18.4 %), arithmetic 19 (15.2 %), bugs in test code 15 (12.0 %), test polluter 2 (1.6 %).

Two things this paper does that most fuzzing papers do not

It publishes a negative result about its own design. Starting from the default configuration — an assumption inherited from essentially all prior configuration fuzzers — turns out not to matter once parameter-aware mutation is present: 68.6 versus 59.2 bugs, and on HBase both variants find 5.6. The paper says why, plainly: random starting points inside the exercised subset “can sometimes even expose paths that exercise more parameters than the default configuration does.” That is a load-bearing finding for anyone building in this space and it would have been easy to omit.

It reports the cost of its prioritisation. Seven bugs found by the weaker tracking variants are missed by the full system, and the paper says why: prioritising seeds that exercise more parameters necessarily

deprioritises seeds that exercise few but still trigger unique bugs. The trade is defended as worthwhile, not hidden.

The triage pipeline is also worth noting on its own: delta-debugging-style randomised binary search to minimise the failure-inducing parameter set, a symptom-based benign-failure filter the authors describe as “neither sound nor complete, but we find them empirically effective”, and grouping by exception type plus minimised inducing set plus project-internal stack trace. Together these “reduced hundreds of thousands of failures to a few hundred groups” — which is the difference between a fuzzer and a usable fuzzer.

Limitations

No dedicated threats section. Applicability requires a uniform GET API — pervasive in mature Java systems but not universal — and an abundance of suitable tests. Evaluation covers one core module per project, with at most 1,000 randomly sampled drivers each. The authors state the external-validity boundary themselves: the conclusions hold “within a fixed time budget with unit-test drivers”, with no claim of extrapolation to arbitrarily long campaigns. Type inference is deliberately simple, taking each parameter’s type from the type of its default value. On projects with small configuration spaces the variants perform almost identically, which is where the single coverage loss appears.

The blackbox cloud-scale fuzzer is not run as a baseline, with a defensible reason: it uses unit tests as a *filter*, discarding any generated configuration that fails one before system testing runs at all — so the failures ConfFuzz targets are, by construction, precisely the ones that tool throws away.

Closing line, verbatim: “Compared with a code-coverage-only greybox baseline that mutates across the full configuration space, ConfFuzz finds substantially more bugs and achieves the highest parameter coverage in all nine systems, exposing 125 configuration-related bugs across all campaigns.”

2.3 · KERNEL FUZZING · Syzkaller spends 57.5% of its executions shrinking inputs; batching the removals cuts that by 60.7% and finds 1.7–2× more bugs 🙌 😊 📄

Efficient Input Minimization for Kernel Fuzzing via Relation- and Type-Guided Optimization

Authors: Hui Guo, Hao Sun, Shan Huang, Ting Su, Geguang Pu, Shaohua Li.

Venue: ACM Transactions on Computer Systems, 2026

Affiliations. East China Normal University and Shanghai Key Laboratory of Trustworthy Computing (Guo, Huang, Su, Pu), ETH Zurich (Sun), The Chinese University of Hong Kong (Li). Journal DOI: <https://doi.org/10.1145/3840388>.

Provenance. The alerted TOCS 2026 article is the extended version of the USENIX ATC 2025 paper “Optimizing Input Minimization in Kernel Fuzzing” (<https://www.usenix.org/conference/atc25/presentation/guo>). The ACM Digital Library PDF returned no readable content this run, so **every number below is from the ATC 2025 conference version**, which was read in full. The journal extension may add material not reflected here; treat the numbers as the conference version’s. Artifact: <https://github.com/ecnusse/SyzMini>.

Licence. The ATC version carries only USENIX’s standard open-access sponsorship banner — no Creative Commons grant and no explicit reuse terms are stated. No figures are reproduced here on that basis.

The measurement that is the contribution

Syzkaller minimises every “interesting” program before adding it to the corpus, removing calls one at a time and simplifying arguments one at a time, re-executing the program after each attempt to check the target call’s coverage survived. The paper asks what that costs, and the answer is the paper:

- Minimisation consumes **57.5 %** of all program executions in a 48-hour campaign, mutation the remaining **42.5 %**.
- It peaks at **68.1 % at the 4.5-hour mark** and never falls back far. The reason is that although interesting programs get rarer over time, they also get more complex — more calls, more arguments — so each one costs more executions to reduce one step at a time.
- Within minimisation, removing calls is **34.0 %** and simplifying arguments **66.0 %**.

The obvious response is refuted first: a Syzkaller variant with minimisation deleted loses **27.5 %** coverage and **40.4 %** of bugs over 48 hours, “and the gap continues to widen with prolonged fuzzing.” So the stage is both indispensable and the single largest consumer of the budget — which is exactly the profile of something worth optimising and exactly the profile of something nobody had profiled.

The two optimisations

Influence-guided call removal replaces one execution per candidate deletion with one execution for a batch. The key relation, defined verbatim: “For two system calls c_i and c_j , c_i has an influence relation with c_j if executing c_i can change the execution path of c_j by modifying the kernel’s internal state that c_j depends on.”

Calls with no influence on the target cannot affect its coverage, so they can all be deleted at once and verified in a single execution. If the batch removal preserves coverage the program is replaced wholesale; otherwise it is rolled back — and either way the sound one-by-one loop still runs over the survivors, so minimality is preserved. The technique is a shortcut layered on top of the exact algorithm, not a replacement for it, which is the whole reason an unsound relation is safe to use.

The relation is not learned. It comes from two free sources. Statically, from the type information already present in Syzkaller’s syscall description language: c_i influences c_j if c_i returns a resource type that c_j consumes, or if c_i has a pointer parameter pointing to a resource with outward data-flow direction that c_j takes inward. Dynamically, from Syzkaller’s own baseline minimisation runs: if removing one call is observed to change the coverage of the call that follows it, an influence edge is recorded. That half was harvested by running until coverage saturated, about four days. Total: **74,865 relations, 44,966 static** and **29,899 dynamic**.

That second source is the neat trick. The unoptimised reducer’s accept/rollback decisions are labelled dependence data that every existing deployment is currently throwing away.

Type-informed argument simplification is a purely static filter: skip fixed-size arguments entirely, spend executions only on variable-size ones. The justification is a reframing of what minimisation is *for*: only those arguments “that truly matter for the subsequent mutations” are worth spending executions on. Trimming superfluous elements from an array meaningfully shrinks the space later mutation must search; defaulting a scalar, the paper argues, yields marginal benefit for the same cost. Minimisation is an investment in future mutation efficiency, not size reduction for its own sake.

This is the one place the paper trades exactness for cost: scalars keep their mutated values, so the output is not argument-minimal by Syzkaller’s standard. It pays because **74–80 %** of syscall parameters are fixed-size across five years of Syzkaller versions.

Results

Linux 5.15, 6.1 and 6.11, KCOV and KASAN enabled, 24-hour campaigns repeated 10 rounds.

	v5.15	v6.1	v6.11	Overall
Syzkaller branches	145.371K	150.677K	133.172K	143.073K
SyzMini branches	164.724K	169.015K	149.311K	161.017K
Improvement	+13.3 %	+12.2 %	+12.1 %	+12.5 %
Speed-up to equal coverage	1.83×	1.61×	1.62×	1.69×

Unique bugs: **50** for SyzMini against **27** for Syzkaller (28 vs 14 on v5.15, 20 vs 12 on v6.1, 12 vs 6 on v6.11) — **1.7–2.0×**. On the 22 bugs both tools found, SyzMini found every one faster and more often. **13 previously unknown bugs** were reported on the latest upstream kernel, all confirmed and four already fixed, spanning Block, Gfs2, NTFS3, Reiserfs, Net, MM/pat, Bpf, Jfs, Usb and Bluetooth.

The controlled replay experiment isolating reduction cost — 16,266 recorded interesting programs replayed under each strategy — is the cleanest result in the paper:

Strategy	Removing calls	Simplifying args	All
one-by-one (baseline)	140,620	260,239	400,859
influence-guided only	60,372 (–57.1 %)	—	320,611 (–20.0 %)
type-informed only	—	96,896 (–62.8 %)	237,516 (–40.7 %)
both	60,372 (–57.1 %)	96,896 (–62.8 %)	157,268 (–60.7 %)

Decoupling the cost claim from end-to-end fuzzing noise by replaying a fixed corpus is a methodological choice worth copying for any “we made an existing stage cheaper” paper.

The budget then shifts where you would want it: on v5.15, minimisation falls from **48.2 % to 11.5 %** of executions while mutation rises from **51.8 % to 88.5 %**. The two strategies are orthogonal, acting on different steps — influence-guided contributes 3.5–4.6 % coverage, type-informed 8.3–9.1 %.

Generality was tested by dropping the optimisations into three other kernel fuzzers selected by a documented protocol (11 papers surveyed from 2019–2024 top venues, 3 excluded as closed-source, 5 as unmaintained): **+14.5 %** coverage and 24 vs 16 bugs for one; **66.7 %** more memory bugs for another; 9 vs 6 bugs reproduced for the third. Total evaluation cost **5,760 machine hours**.

The sensitivity sweep is the reason to trust the approach: supplying only **60 %** of the influence relations already saturates the benefit, so the technique degrades gracefully as the relation database ages against a moving kernel.

Formal notation. The paper contains no numbered display equations and no theorem, lemma or complexity bound. Its formal content is the influence-relation definition quoted above, two pseudocode algorithms, and the coverage-preservation predicate. None is invented here.

Limitations

No dedicated limitations section. The generality claim beyond the three evaluated fuzzers is explicitly a belief. The dynamic half of the relation database cost four days to build and is tied to a kernel and a Syzkaller version. The RQ4 bug-reproduction dataset was cut from 100 bugs to the first 10 for cost. And one gap is worth naming because the neighbouring paper in this window does it differently: randomness is addressed with 10 rounds and ± 1 s.d. shading, but **no statistical significance test is reported** — no Mann-Whitney U, no $\hat{A}_{12}$ — despite the effect sizes being large enough that a test would almost certainly have passed.

Closing line, verbatim: “We believe our optimization strategies are general and could benefit many other kernel fuzzers.”

Also Surfaced

3.1 · VULN DETECTION · A shortcut-learning critique from inside the saturated DL-for-vulnerability-detection literature: models keying on security keywords rather than code semantics 

Mitigating Keyword Bias in Java Vulnerability Detection through Dual-Stream CodeBERT with Security Feature Engineering

Authors: A. Khurana, T. Farasat, J. Posegga, F. Lemmerich.

Venue: CEUR-WS Vol-4238, 2026

Link. <https://ceur-ws.org/Vol-4238/paper1.pdf>

Provenance. Alert-snippet read only; not fetched.

Deep-learning vulnerability detection is the most saturated area this digest sees, and it is normally screened out at Stage 1. This one is listed because its stated premise is a critique of that literature rather than another entry in it: models “learn to associate the presence of security-related keywords with vulnerability labels” instead of learning the code semantics that actually separate vulnerable from secure.

That is a shortcut-learning claim, and it is the same failure mode that the strongest paper in yesterday’s window identified in a different guise — agents leaning on symbol names rather than instruction-level analysis, such that symbol stripping cost more than compiler optimisation. When a measurement instrument keys on a lexical proxy that correlates with the label, high reported accuracy tells you about the proxy, not the capability. That pattern is worth tracking wherever it appears.

The proposed remedy — a dual-stream CodeBERT with hand-engineered security features — is a more conventional contribution, and a workshop venue with a Java-only scope limits how far any result would generalise. The question that would make it worth a full read is whether the paper *quantifies* the bias with an intervention (does performance collapse when keywords are masked or adversarially inserted?) or merely asserts it before proposing an architecture. A convincing measurement of the bias would be more valuable than the architecture, and would be reusable by everyone working on these benchmarks. The snippet does not say which it is. Listed for the reader’s own triage.

Cross-Paper Synthesis

All three deep-read papers won by profiling, not by inventing. SyzMini’s entire contribution rests on a measurement nobody had published — that 57.5 % of a kernel fuzzing campaign goes to input reduction. ConfFuzz’s decisive number is that unit-test drivers run at 20.57 exec/s against system testing’s 0.0371, so the technique’s cleverness sits on top of a 550× throughput change obtained by choosing a different driver. CVE-Genie’s \$2.77 per CVE comes not from a better prompt but from noticing that developer agents burn turns and critics do not, and assigning models accordingly — the same pipeline with reasoning models as developers costs \$10–20 per CVE. In each case the intellectual work was identifying which resource was actually being consumed. That is an unglamorous research strategy with an unusually good hit rate, and it is available to anyone willing to instrument their own pipeline before optimising it.

Put the imprecise component where being wrong costs time, not correctness. This is the strongest shared design pattern in the window and it is worth stating as a rule. SyzMini’s influence relation is an unsound, incomplete approximation assembled from static types and scavenged logs — and it is safe precisely because a wrong guess causes a rolled-back batch followed by the exact one-by-one loop, so imprecision costs executions and never minimality. ConfFuzz’s benign-failure filter is described by its authors as “neither sound nor complete”, and it only decides what a human sees. CVE-Genie’s critics are deliberately biased toward rejection, so the system produces false negatives rather than corrupt dataset entries. The general form: use approximate analysis to *order, batch and triage*, never to *decide*, and you get most of the speed with none of the soundness risk. This is also the shape of the argument for where an LLM belongs inside a correctness-critical loop.

Ablations are now doing the persuading, and the strongest ones are destructive. CVE-Genie’s monolithic-agent row reads 0/15 against 15/15. ConfFuzz’s component matrix walks 19.4 → 59.2 → 68.6 → 94.8 bugs. SyzMini’s α/β split shows the two strategies are orthogonal and quantifies each. Yesterday’s window made the same observation from a benchmark-construction angle: the ablation proving the cheap alternative collapses is what converts an expensive design from a liability into the central claim. Two consecutive windows in which the most valuable experiment in the paper is the one that attacks the paper’s own design is a pattern, not a coincidence.

Statistical rigour in fuzzing evaluation has become bimodal, in the same subfield in the same year. ConfFuzz reports five repetitions, 95 % confidence intervals on every mean, paired *t*-tests with exact *p*-values, Cohen’s *d_z* effect sizes, and a full VM-hour accounting — 48,380 — plus an explicit statement that its conclusions are budget-bounded. SyzMini reports 10 rounds and ± 1 s.d. shaded regions with the observation that “the overlap between the shaded areas is small”, and no test at all. Both are strong papers with large true effects. But readers cannot tell large-and-verified from large-and-asserted without doing the work themselves, and the cost of the verified version is now demonstrably low.

Oracle construction remains the recurring bottleneck, and reuse keeps beating authorship. CVE-Genie’s CTF flag turns a heterogeneous success condition into one uniform pass/fail interface. ConfFuzz reuses developers’ existing unit tests as both driver and oracle. SyzMini reuses the target call’s branch coverage as its preservation predicate. Yesterday’s window produced two papers arguing that hand-written per-instance oracles are a systematic source of inflated scores, and that reusing an existing project’s own tests fixes it structurally. Five papers across two days, all converging on the same conclusion: the oracle you write yourself is the component most likely to be wrong, and the cheapest fix is to derive it from something that already exists and was not written to be graded against.

A note on what “the agent did it” now means. CVE-Genie synthesised working exploits for 428 real vulnerabilities, including 46 % of successes where no PoC existed to copy. It also scores 0 % on OS command injection without a PoC and 8 % on blockchain projects, and 198 of 413 failures simply ran out of budget. The capability is real, extremely uneven across domains, and bounded by cost rather than by reasoning in a large fraction of cases. Anyone reading a headline agent-capability number should ask which of the three is doing the work.

Writing & Rationale Insights

Make the profiling result the paper. SyzMini’s abstract leads with a cost measurement — over half the fuzzing resources — rather than with a technique. That single number reframes a stage everyone treated as overhead into a target, and it makes the two optimisations that follow feel inevitable rather than incremental. If your contribution is “we made an existing stage cheaper”, the measurement establishing that the stage was expensive is not background; it is the argument.

Present the algorithm as a diff. ConfFuzz gives Algorithm 1 as the standard coverage-guided loop and Algorithm 2 as its own, with the changed lines highlighted and annotated. A reader who knows greybox fuzzing can absorb the entire contribution in thirty seconds and knows precisely what was and was not changed. Compare this with the common alternative of presenting a full novel algorithm and leaving the reader to diff it mentally against a baseline they have to reconstruct.

Run the ablation that could embarrass you, and lead with the result. CVE-Genie’s monolithic-agent row is the strongest possible statement that the architecture is load-bearing, and it exists only because the authors tested the objection a sceptical reviewer would raise first. The same paper reports that removing critics raises *false* reproductions by 47 % — a number that quantifies the risk in its own output. Both make the paper more credible, not less.

Publish the design decision that turned out not to matter. ConfFuzz shows that starting from the default configuration, inherited from essentially all prior work in the area, is worth almost nothing once parameter-aware mutation is present, and explains why. Negative results about inherited assumptions are among the highest-value-per-word content a paper can carry, and they cost nothing but the willingness to report a flat comparison.

Choose an ablation baseline that saturates. CVE-Genie deliberately runs ablations on 15 CVEs the full system solves 15/15, calling it “a controlled probe”. Because the ceiling is reached, every drop is unambiguously caused by the manipulation rather than by task difficulty. This is a small design choice that removes an entire class of reviewer objection.

Decouple mechanism cost from end-to-end noise. SyzMini’s replay experiment — 16,266 recorded programs, replayed under each reduction strategy, counting executions — measures the thing being claimed without the variance of a live fuzzing campaign, and then reports end-to-end results separately. Mechanism cost, end-to-end effect, ablation, portability: that four-part structure is a reusable template.

Report your prioritisation’s losses. ConfFuzz notes that seven bugs found by weaker variants are missed by the full system, because prioritising breadth deprioritises seeds that exercise few parameters. Stating the trade and defending it is more persuasive than a table in which the proposed method dominates every cell, which readers have learned to distrust.

On this report’s own limits. All three deep-reads are genuine full-text reads and every number is the paper’s own. The exception is entry 2.3, where the alerted artefact is the TOCS 2026 journal extension

but the readable text was the ATC 2025 conference version; that substitution is stated in the entry rather than papered over, and the journal version may contain material not reflected here. No figures are embedded: entry 2.3's source grants no reuse licence, and for the two CC BY 4.0 papers the raster-extraction tooling was unavailable this run. No formal notation appears for 2.1 or 2.3 because those papers contain none, and the three formulas quoted for 2.2 are the paper's own, cited by section because it does not number them.